

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1
По дисциплине: «Аппаратно-программное обеспечения ЭВМ и сетей»

Выполнил:
Студент 4-го курса
Группы АС-66
Николова М.С.
Проверил:
Кулик А.Д.

Цель работы: изучение программной модели микропроцессора, регистров, организации памяти; практическая работа с ассемблерными вставками

Вариант 1

Ход работы

1. Изучить следующие теоретические сведения: программная модель микропроцессора; регистры и их назначение; организация памяти; основные конструкции и базовые команды ассемблера (используя материалы лекций и методические указания).
2. Написать ассемблерную вставку, реализующую обработку строки согласно варианту, выданному преподавателем. Оформить ее в виде отдельной функции.
3. Реализовать данную обработку строки также в виде функции на C++. Сравнить быстродействие обоих вариантов. В отчете отразить выводы.

Содержание отчета:

- титульный лист;
- описание варианта задания;
- текст программы;
- скриншоты результатов.

ПРИМЕЧАНИЕ: для разработки использовать MS Visual Studio.

Вариант задания

1. Перевернуть строку.

```
#include <iostream>
#include <cstring>
#include <chrono>
#include <iomanip>
#include <algorithm>

using namespace std;

void ReverseStringCPP(char* str) {
    if (str) {
        reverse(str, str + strlen(str));
    }
}

void ReverseStringASM(char* str) {
    if (!str) return;

    __asm {

        push esi
        push edi
        push eax
        push ebx

        mov esi, str
        mov edi, str

        mov al, 0
        find_null:
        cmp byte ptr[edi], al
        je found_null
```

```

        inc edi
        jmp find_null

    found_null :
dec edi

    reverse_loop :
cmp esi, edi
jge done

    mov al, byte ptr[esi]
    mov bl, byte ptr[edi]
    mov byte ptr[esi], bl
    mov byte ptr[edi], al

    inc esi
    dec edi
    jmp reverse_loop

done :

    pop ebx
    pop eax
    pop edi
    pop esi
}
}

```

```

int main() {
    setlocale(LC_ALL, "Russian");

    char str1[] = "Hello, World!";
    char str2[] = "Hello, World!";

    cout << "Исходная строка: " << str1 << endl;

    ReverseStringCPP(str1);
    cout << "Результат (C++): " << str1 << endl;

    ReverseStringASM(str2);
    cout << "Результат (ASM): " << str2 << endl;

    cout << "\nТестирование быстродействия" << endl;

    const size_t N = 10'000'000;
    const int ITERATIONS = 100;

    char* largeStrCPP = new char[N + 1];
    char* largeStrASM = new char[N + 1];
    for (size_t i = 0; i < N; ++i) {
        char ch = 'a' + (i % 26);
        largeStrCPP[i] = ch;
        largeStrASM[i] = ch;
    }
    largeStrCPP[N] = '\0';
    largeStrASM[N] = '\0';

    auto startCPP = chrono::high_resolution_clock::now();
    for (int i = 0; i < ITERATIONS; ++i) {
        ReverseStringCPP(largeStrCPP);
    }
    auto endCPP = chrono::high_resolution_clock::now();
    chrono::duration<double, milli> durationCPP = endCPP - startCPP;

    auto startASM = chrono::high_resolution_clock::now();
    for (int i = 0; i < ITERATIONS; ++i) {

```

```

        ReverseStringASM(largeStrASM);
    }
    auto endASM = chrono::high_resolution_clock::now();
    chrono::duration<double, milli> durationASM = endASM - startASM;

    cout << fixed << setprecision(3);
    cout << "Время выполнения C++ (" << ITERATIONS << " итераций): " << durationCPP.count() <<
" ms" << endl;
    cout << "Время выполнения ASM (" << ITERATIONS << " итераций): " << durationASM.count() <<
" ms" << endl;

    delete[] largeStrCPP;
    delete[] largeStrASM;

    return 0;
}

```

```

Исходная строка: Hello, World!
Результат (C++): !dlroW ,olleH
Результат (ASM): !dlroW ,olleH

```

```

Тестирование быстродействия
Время выполнения C++ (100 итераций): 1569.778 ms
Время выполнения ASM (100 итераций): 772.679 ms

```

Вывод: я изучила программную модель микропроцессора, регистров, организации памяти; выполнила практическую работу с ассемблерными вставками